

TRƯỜNG ĐẠI HỌC KIẾN TRÚC ĐÀ NẴNG (DAU)

Chương 5: Kỹ thuật lập trình mạng

Bài giảng · Chương 5

Học thật · làm thật · code thật

TS. Đỗ Phúc Hảo – Giảng viên *Lecturer*

Khoa Công nghệ Thông tin, Trường Đại học Kiến trúc Đà Nẵng

haodp.ai@gmail.com · 0365 674 592 · Đà Nẵng, 2026

Nội dung buổi học *Lecture outline*

1. **Mô hình client-server và nền tảng mạng** *client-server, IP, port*
2. **Lập trình socket TCP với Java** *ServerSocket, Socket*
3. **Phục vụ nhiều máy khách bằng đa luồng** *thread-per-connection*
4. **Tuần tự hóa và truyền đối tượng** *serialization*
5. **Giới thiệu UDP với DatagramSocket** *connectionless*

Slide tiếng Việt là chính; thuật ngữ tiếng Anh in nghiêng; code Java comment tiếng Anh. Chương này dùng lại đa luồng của chương 4 như một công cụ.

1

Mô hình client-server

IP, cổng, TCP vs UDP

Vì sao cần lập trình mạng *Why network programming*

- Một chương trình hữu ích hiếm khi sống một mình: nó cần nói chuyện với chương trình khác, cùng máy hay ở đầu kia thế giới.
- *Network programming* là kỹ thuật cho hai chương trình trao đổi dữ liệu qua hạ tầng mạng.
- Java có thư viện mạng gọn trong gói `java.net`, che giấu phần lớn chi tiết giao thức.

GHI NHỚ · KEY POINT

Tập trung vào logic ứng dụng; để `java.net` lo phần truyền dữ liệu qua mạng.

Mô hình client-server *Client-server model*

DAU

ĐỊNH NGHĨA · DEFINITION

Máy chủ *server* chờ và đáp ứng yêu cầu ở một địa chỉ cố định; **máy khách** *client* chủ động khởi tạo kết nối và gửi yêu cầu.

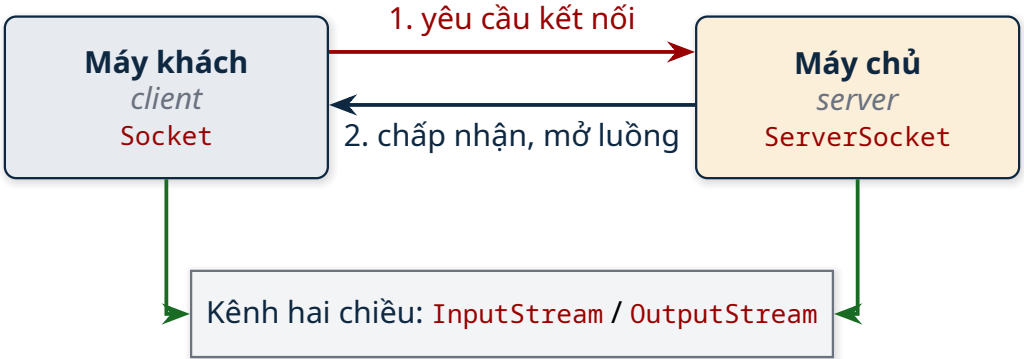
Máy chủ *server*
thụ động chờ tại địa chỉ ai cũng biết

Máy khách *client*
chủ động tìm tới, gửi yêu cầu

Ví dụ: trình duyệt là máy khách, máy chủ web là máy chủ. Ở mỗi kết nối, vai trò luôn rõ ràng dù một chương trình có thể đóng cả hai vai.

Sơ đồ client-server qua socket *Client-server over socket*

DAU



qua mạng IP, định danh bằng cặp (địa chỉ, cổng)

Địa chỉ IP và cổng *IP address and port*

- **Địa chỉ IP** *IP address*: định danh máy. IPv4 bốn số 0..255, vd `192.168.1.10`; `127.0.0.1` (`localhost`) trỏ về chính máy.
- **Cổng** *port*: số 16 bit 0..65535, đánh dấu từng dịch vụ trên cùng một máy.
- Cặp `(IP, cổng)` = một **điểm cuối** *endpoint*, xác định duy nhất một đầu giao tiếp.

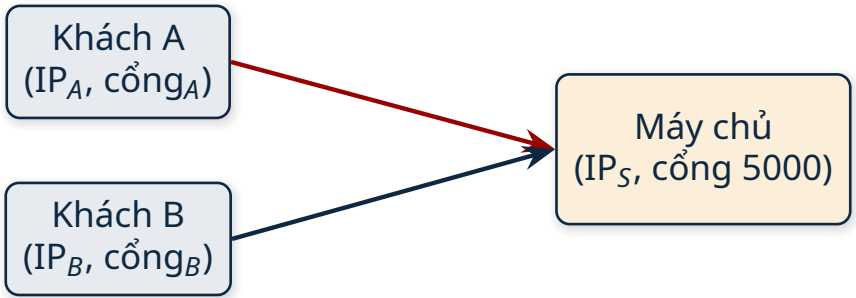
Cổng dưới 1024 dành cho dịch vụ chuẩn (80 HTTP, 443 HTTPS, 22 SSH). Khi thử nghiệm, chọn cổng cao như 5000 hay 8080.

Bốn yếu tố của một kết nối *Four-tuple of a connection*

DAU

GHI NHỚ · KEY POINT

Một kết nối xác định bởi bốn yếu tố: (IP khách, cổng khách) và (IP chủ, cổng chủ). Nhờ vậy nhiều máy khách cùng nối tới một cổng máy chủ mà hệ điều hành vẫn phân biệt được.



TCP và UDP: hai triết lý *TCP vs UDP*

DAU

TCP *Transmission Control Protocol*
hướng kết nối; bắt tay trước; dòng byte tin cậy, đúng thứ tự, gửi lại khi mất. Nặng hơn.

UDP *User Datagram Protocol*
phi kết nối; gửi gói độc lập; có thể mất, đảo, trùng. Nhẹ và nhanh.

TCP hợp truyền tệp, web, thư (mất một byte là hỏng). UDP hợp phát trực tuyến, gọi thoại, game (dữ liệu mới quan trọng hơn dữ liệu cũ).

Bảng so sánh TCP và UDP *Comparison table*

DAU

Tiêu chí	TCP	UDP
Kiểu liên lạc	Hướng kết nối, bắt tay trước	Phi kết nối, gửi ngay
Độ tin cậy	Bảo đảm tới nơi, gửi lại khi mất	Không bảo đảm, có thể mất gói
Thứ tự dữ liệu	Đúng thứ tự gửi	Có thể đảo thứ tự
Đơn vị dữ liệu	Dòng byte liên tục *stream*	Gói rời rạc *datagram*
Chi phí, độ trễ	Cao hơn	Thấp, nhẹ
Lớp Java	`Socket`, `ServerSocket`	`DatagramSocket`, `DatagramPacket`
Ứng dụng	Web, truyền tệp, thư	Phát trực tuyến, gọi thoại, game

TCP là mặc định an toàn; UDP giới thiệu ngắn ở cuối.



2

Socket TCP

ServerSocket & Socket

Khái niệm socket *Socket*

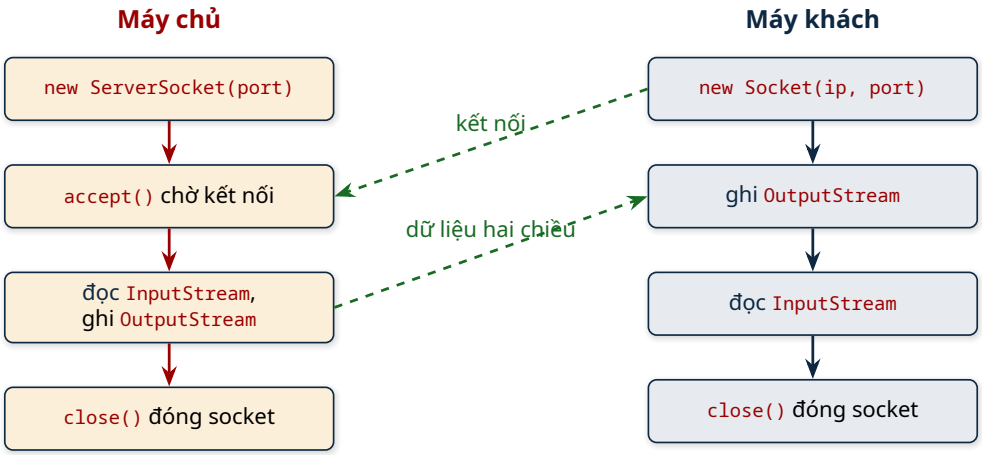
ĐỊNH NGHĨA · DEFINITION

Socket là một đầu mối giao tiếp ở mức phần mềm, đại diện cho một điểm cuối của kết nối. Qua socket, chương trình đọc và ghi dữ liệu y như đọc ghi tệp.

- `ServerSocket`: phía máy chủ, gắn vào một cổng và **lắng nghe** *listen* yêu cầu đến.
- `Socket`: một kết nối đã thiết lập, dùng ở cả hai phía.
- Từ mỗi `Socket`: `getInputStream` để đọc, `getOutputStream` để ghi.

Trình tự gọi của một phiên TCP *TCP session flow*

DAU



Máy chủ echo *Echo server*

DAU

- **Máy chủ echo** *echo server*: nhận một dòng văn bản rồi gửi trả lại đúng dòng đó.
- Bài tập kinh điển: nhỏ nhưng đủ thể hiện toàn bộ vòng đời một kết nối.
- Phiên bản đầu chỉ phục vụ một máy khách tại một thời điểm.

GHI NHỚ · KEY POINT

`server.accept()` **chặn** *block* luồng cho tới khi có máy khách kết nối, rồi trả về một `Socket` riêng cho kết nối đó.

Mã: máy chủ echo TCP (một máy khách) *Echo server code*

DAU

```java
int port = 5000;
// ServerSocket binds to the port and listens for clients
try (ServerSocket server = new ServerSocket(port)) {
   System.out.println("Server listening on port " + port);
   // accept() blocks until a client connects, returns its Socket
   try (Socket client = server.accept()) {
      BufferedReader in = new BufferedReader(
            new InputStreamReader(client.getInputStream()));
      PrintWriter out = new PrintWriter(
            client.getOutputStream(), true); // true = auto-flush
      String line;
      while ((line = in.readLine()) != null) { // null = client closed
         out.println("ECHO: " + line); // send the line back
      }
   }
}
```

Giải thích máy chủ echo *Echo server explained*

- `InputStreamReader` + `BufferedReader`: biến luồng byte vào thành luồng ký tự đọc theo dòng.
- `PrintWriter` với `true`: bật **tự đẩy dữ liệu** *auto-flush* mỗi lần xuống dòng.
- Vòng lặp dừng khi `readLine` trả `null`, dấu hiệu máy khách đã đóng kết nối.

LỖI HAY GẶP · PITFALL

Quên `true` thì dữ liệu có thể nằm trong bộ đệm chưa thực sự gửi đi. Quên đóng socket sẽ dần cạn cổng và file descriptor; hãy mở trong `try-with-resources`.

Mã: máy khách echo TCP *Echo client code*

DAU

```
String host = "localhost"; // server address, same machine here
int port = 5000;
try (Socket socket = new Socket(host, port)) { // connect to server
   BufferedReader in = new BufferedReader(
         new InputStreamReader(socket.getInputStream()));
   PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
   BufferedReader keyboard = new BufferedReader(
         new InputStreamReader(System.in));
   String userLine;
   while ((userLine = keyboard.readLine()) != null
            && !userLine.isEmpty()) {
      out.println(userLine); // send to server
      String reply = in.readLine(); // wait for the echo
      System.out.println("Server says: " + reply);
   }
}
```

Chạy thử và đo độ trễ *Run and measure*

DAU

- Mở hai cửa sổ dòng lệnh: một bên chạy `EchoServer`, một bên `EchoClient`.
- Cùng một máy dùng `localhost`; hai máy khác trong mạng nội bộ thì thay `host` bằng IP thật của máy chủ.

Có thể đo độ trễ khứ hồi *round-trip time*: ghi mốc thời gian quanh một lượt gửi và nhận, in hiệu số. Nhỏ khi nội bộ, tăng rõ khi máy chủ ở xa.

3

Đa luồng cho máy chủ

thread-per-connection

Hạn chế của máy chủ một máy khách *The single-client limit*

- Sau khi `accept` một máy khách, máy chủ mắc kẹt trong vòng lặp phục vụ máy khách ấy.
- Mọi máy khách khác phải xếp hàng chờ. Một máy chủ thực tế phải phục vụ song song.

GHI NHỚ · KEY POINT

Mẫu hình **một luồng cho mỗi kết nối** *thread-per-connection*: luồng chính chỉ lặp `accept`, mỗi máy khách mới được giao cho một luồng riêng. Luồng chính không bị một máy khách chậm làm tắc nghẽn.

Mã: máy chủ echo đa luồng (luồng chính) *Multi-client server, main*

DAU

```java
int port = 5000;
// a thread pool reuses workers instead of one thread per client
ExecutorService pool = Executors.newFixedThreadPool(20);
try (ServerSocket server = new ServerSocket(port)) {
   System.out.println("Multi-client server on port " + port);
   while (true) { // keep accepting forever
      Socket client = server.accept(); // blocks until a client arrives
      pool.submit(new ClientHandler(client)); // hand off to a worker
   }
}
```

Luồng chính lặp vô hạn, mỗi vòng chỉ `accept` rồi giao `Socket` cho một tác vụ chạy trên luồng của bể luồng *thread pool*, rồi quay lại `accept` ngay.

Mã: ClientHandler phục vụ một kết nối *ClientHandler*

DAU

```java
// Each connection is served by its own task on a pool thread.
class ClientHandler implements Runnable {
   private final Socket socket;
   ClientHandler(Socket socket) { this.socket = socket; }
   @Override
   public void run() {
      try (Socket s = socket) { // ensures socket is closed
         BufferedReader in = new BufferedReader(
               new InputStreamReader(s.getInputStream()));
         PrintWriter out = new PrintWriter(s.getOutputStream(), true);
         String line;
         while ((line = in.readLine()) != null)
            out.println("ECHO: " + line);
      } catch (IOException e) {
         System.err.println("Client error: " + e.getMessage());
      }
   }
}
```

Vì sao dùng bể luồng *Why a thread pool*

- `ExecutorService` tái dùng một số luồng cố định thay vì `new Thread` cho mỗi máy khách.
- Tránh chi phí tạo luồng liên tục và tránh việc quá nhiều máy khách sinh hàng nghìn luồng làm sập máy chủ.
- Logic phục vụ một máy khách nằm gọn trong `run` của `Runnable`.

LỖI HAY GẶP · PITFALL

Nhiều luồng cùng ghi vào tài nguyên chung (danh sách trực tuyến, bộ đếm) lại gặp **tranh chấp** *race condition* của chương 4. Dùng `ConcurrentHashMap` hoặc đồng bộ hóa. Từng socket chỉ một luồng dùng nên không cần khóa.



4

Tuần tự hóa

Serializable & ObjectStream

Khái niệm tuần tự hóa *Serialization*

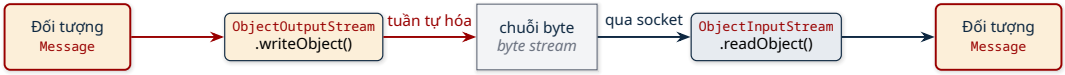
ĐỊNH NGHĨA · DEFINITION

Tuần tự hóa *serialization* biến trạng thái của một đối tượng thành chuỗi byte để lưu trữ hay truyền đi. Quá trình ngược lại gọi là **giải tuần tự hóa** *deserialization*.

- Nhiều ứng dụng cần gửi cả một đối tượng có cấu trúc, vd bản tin (người gửi, nội dung, thời điểm).
- Lớp chỉ cần cài giao tiếp đánh dấu `Serializable` *marker interface*: không có phương thức.
- `ObjectOutputStream` ghi nguyên đối tượng, `ObjectInputStream` đọc và dựng lại y nguyên.

Luồng dữ liệu khi tuần tự hóa *Serialization data flow*

DAU



máy gửi → mạng → máy nhận, đối tượng được dựng lại y nguyên

Mã: lớp dữ liệu Serializable *Serializable data class*

DAU

```java
public class Message implements Serializable {
   private static final long serialVersionUID = 1L; // version guard
   private final String sender;
   private final String content;
   private final LocalDateTime sentAt;
   private transient String tempToken; // not serialized, restored as null
   public Message(String sender, String content) {
      this.sender = sender;
      this.content = content;
      this.sentAt = LocalDateTime.now();
   }
   @Override
   public String toString() {
      return "[" + sentAt + "] " + sender + ": " + content;
   }
}
```

serialVersionUID và transient *Version id and transient*

- `serialVersionUID`: số phiên bản gắn với lớp. Khi giải tuần tự hóa, Java so số này; lệch thì báo lỗi để tránh dựng nhầm từ phiên bản không tương thích.
- `transient`: đánh dấu trường không truyền đi (mật khẩu tạm, giá trị tính lại được); khi dựng lại nhận giá trị mặc định.

LỖI HAY GẶP · PITFALL

Mọi trường không `transient` cũng phải `Serializable`, nếu không Java ném `NotSerializableException`. Đừng giải tuần tự hóa dữ liệu từ nguồn lạ: đây là một lỗ hổng an ninh đã biết.

Mã: gửi và nhận một đối tượng *Send and receive an object*

DAU

Máy khách gửi

```
try (Socket socket =
       new Socket("localhost", 5000);
     ObjectOutputStream out =
       new ObjectOutputStream(
         socket.getOutputStream())) {
    Message msg = new Message(
       "Hao", "Xin chao");
    out.writeObject(msg); // serialize + send
    out.flush();
}
```

Máy chủ nhận

```
try (ServerSocket server =
       new ServerSocket(5000);
     Socket client = server.accept();
     ObjectInputStream in =
       new ObjectInputStream(
         client.getInputStream())) {
    // readObject -> Object, then cast
    Message msg =
       (Message) in.readObject();
    System.out.println(msg);
}
```

Cùng lớp ở hai phía *Same class on both sides*

- Phía nhận khai báo ném thêm `ClassNotFoundException`: máy ảo cần tìm thấy định nghĩa lớp `Message` trong classpath khi dựng lại đối tượng.
- Cả máy gửi và máy nhận đều phải có cùng lớp `Message`.
- Thay đổi cấu trúc lớp ở một phía mà phía kia không cập nhật sẽ làm hỏng giao tiếp.

GHI NHỚ · KEY POINT

Tuần tự hóa Java tiện khi hai bên cùng là Java và chia sẻ lớp. Liên thông với hệ thống ngôn ngữ khác thì chọn định dạng trung lập như JSON hay Protocol Buffers.



5

UDP

DatagramSocket & DatagramPacket

UDP: nhẹ và phi kết nối *UDP basics*

- Khi không cần độ tin cậy của TCP mà cần sự nhẹ nhàng, ta dùng UDP.
- `DatagramSocket` để gửi nhận; `DatagramPacket` là một gói dữ liệu kèm đích đến.
- Không có khái niệm kết nối hay luồng: đóng dữ liệu vào gói, ghi rõ địa chỉ và cổng nhận, rồi gửi.

Bên gửi chỉ `send` rồi quên: nếu gói lạc đường, không có thông báo. Bên nhận chuẩn bị một mảng đệm đủ lớn vì mỗi gói UDP có giới hạn kích thước.

Mã: gửi và nhận một gói UDP *UDP send and receive*

DAU

Bên gửi

```
try (DatagramSocket socket =
      new DatagramSocket()) {
   byte[] data = "Hello UDP".getBytes();
   InetAddress addr =
      InetAddress.getByName("localhost");
   DatagramPacket packet =
      new DatagramPacket(
        data, data.length, addr, 6000);
   socket.send(packet); // fire and forget
}
```

Bên nhận

```
try (DatagramSocket socket =
      new DatagramSocket(6000)) { // bind
   byte[] buffer = new byte[1024];
   DatagramPacket packet =
      new DatagramPacket(
        buffer, buffer.length);
   socket.receive(packet); // blocks
   String text = new String(
     packet.getData(), 0, packet.getLength())
          ;
}
```

Tư duy UDP so với TCP *UDP vs TCP mindset*

DAU

UDP
không bắt tay, không luồng liên tục; người lập trình tự lo độ tin cậy nếu cần

TCP
bắt tay, dòng byte tin cậy; mặc định an toàn cho ứng dụng phổ thông

Sự đơn giản của UDP đổi lấy việc tự đánh số gói và yêu cầu gửi lại nếu ứng dụng cần. Vì thế với phần lớn ứng dụng, TCP vẫn là lựa chọn mặc định.

6

Tóm tắt & bài tập

summary & homework

Tóm tắt chương *Chapter summary*

- Mô hình client-server: một bên thụ động chờ ở địa chỉ cố định, một bên chủ động tìm tới; cặp `(IP, cổng)` định danh từng đầu giao tiếp.
- TCP tin cậy mà nặng; UDP nhẹ mà không bảo đảm.
- Socket TCP qua `ServerSocket` và `Socket`: đọc ghi mạng như đọc ghi tệp.
- Đa luồng một luồng cho mỗi kết nối: phục vụ nhiều máy khách cùng lúc.
- Tuần tự hóa với `Serializable` + `ObjectStream`: gửi nguyên một đối tượng; `transient` loại trường không truyền.

GHI NHỚ · KEY POINT

TCP là mặc định an toàn; `DatagramSocket` mở cánh cửa UDP cho ứng dụng ưu tiên tốc độ.

Bài tập về nhà *Homework*

1. Giải thích vì sao cặp `(IP, cổng)` là cần thiết; cho một ví dụ một máy chạy hai dịch vụ trên hai cổng.
2. Lập bảng so sánh TCP và UDP theo độ tin cậy, thứ tự, chi phí; mỗi giao thức nêu một ứng dụng phù hợp.
3. Mở rộng máy chủ echo đa luồng thành máy chủ chuyển chữ hoa; viết kèm máy khách thử nghiệm.
4. Thêm trường `priority` và một trường `transient` vào `Message`; ghi ra tệp rồi đọc lại, xác nhận `transient` về giá trị mặc định.

Bài tập về nhà (tiếp) *Homework, continued*

5. Xây phòng chat nhóm trên TCP: máy chủ giữ danh sách máy khách trong `ConcurrentHashMap`, phát lại tin cho mọi máy khách khác *broadcast*. Giải thích vì sao cần cấu trúc an toàn luồng.
6. Chuyển ví dụ đối tượng sang gửi `List<Message>` thay vì một đối tượng đơn; rút ra nhận xét về tuần tự hóa các đối tượng lồng nhau.
7. Lập luận điều gì xảy ra nếu phía gửi gửi liên tiếp một nghìn gói UDP mà phía nhận xử lý chậm; so sánh với TCP.

Bài (3), (5), (6) là bài lập trình; nộp mã nguồn kèm mô tả cách chạy.

CẢM ƠN

Hỏi đáp · Thảo luận

TS. Đỗ Phúc Hảo

haodp.ai@gmail.com · 0365 674 592 · Khoa CNTT, ĐH Kiến trúc Đà Nẵng